

PRUEBA TÉCNICA

Data Platform Engineer Lead

Documento de Diseño y Arquitectura

Cliente: TUMIPAY | Proceso: Revolutiva

Candidato

Ronald Arévalo

Data Platform Engineer Lead | AWS Certified Database – Specialty
319.762.26.35 | ronald.jose.arevalo@gmail.com | Bogotá, Colombia

Versión 1.0 | Mayo 2026

Tabla de contenido

1. Resumen ejecutivo	4
2. Contexto y entendimiento del caso	5
2.1 Problema de negocio	5
2.2 Fuentes de información	5
2.3 Supuestos de negocio adoptados	5
3. Hallazgos de calidad de datos	6
3.1 Inventario de hallazgos	6
3.2 Estrategia de tratamiento	6
4. Arquitectura de la solución	7
4.1 Visión general	7
4.2 Componentes por capa	7
4.3 Justificación de decisiones clave	8
4.4 Estructura del repositorio	10
5. Modelo dimensional	11
5.1 Visión del modelo estrella	11
5.2 Definición de dimensiones	11
5.3 Definición de hechos	13
5.4 Granularidad y aditividad	14
6. KPIs de negocio	14
6.1 Catálogo de KPIs	14
7. Capa BI: vistas y consultas analíticas	17
7.1 Vistas analíticas (contrato BI)	17
7.2 Consultas analíticas	17
8. Capa BI: modelo semántico Power BI	18
8.1 Modelo semántico propuesto	18
8.2 Medidas DAX principales	19
8.3 Diseño conceptual del dashboard	19
8.4 Seguridad y acceso (RLS)	20
9. DBA, performance y seguridad	21
9.1 Estrategia de índices	21
9.2 Roles y permisos (principio de menor privilegio)	21
9.3 Tratamiento de datos sensibles (PII)	22
9.4 Backup y restore	22
9.5 Auditoría y trazabilidad	23
9.6 Performance — consultas críticas y optimización	23
9.7 Atomicidad transaccional de la carga al DWH	23
9.8 CI/CD con GitHub Actions	24
10. Evolución a producción en AWS	25

10.1	Adopción del patrón Medallion (Bronze / Silver / Gold)	25
10.2	Arquitectura objetivo en AWS	26
10.3	Transformaciones con dbt: dónde aplica y dónde no	26
10.4	Gobierno de datos: Lake Formation y AWS DataZone	30
10.5	Mapeo componente local → AWS	31
10.6	Diseño Redshift específico (capa Gold)	32
10.7	Seguridad en AWS	33
10.8	Escalabilidad y decisiones de capacidad	33
11.	Gobierno de datos y calidad	34
11.1	Reglas de calidad implementadas	34
11.2	Modelo de la tabla dq_errors	35
11.3	Métricas de calidad	35
11.4	Lineage / linaje de datos	35
12.	Diccionario de datos del modelo final	36
12.1	Resumen de tablas	36
13.	Plan de ejecución del proyecto	37
13.1	Cronograma propuesto	37
13.2	Definición de 'hecho' (Definition of Done)	37
14.	Mejoras futuras y limitaciones reconocidas	38
14.1	Limitaciones de la entrega actual	38
14.2	Roadmap evolutivo (post-entrega)	38
14.3	Riesgos identificados	39

1. Resumen ejecutivo

Este documento presenta el diseño técnico de una solución de datos para TUMIPAY, una fintech que requiere consolidar información dispersa de clientes, créditos y pagos en una fuente confiable, generar indicadores de negocio y dejar una base preparada para evolucionar hacia una plataforma productiva en AWS.

La solución propuesta sigue un enfoque pragmático: una arquitectura por capas (Raw → Staging → Data Warehouse → BI) implementada localmente con Python y PostgreSQL, lista para ejecutarse con Docker Compose, y acompañada de una propuesta de evolución cloud sobre AWS que adopta el patrón Medallion (Bronze/Silver/Gold), transformaciones declarativas con dbt-redshift y gobierno federado opcional con AWS DataZone cuando el volumen, la criticidad o la concurrencia lo justifiquen.

El diseño prioriza tres principios:

- **Criterio sobre tamaño.** No se sobredimensiona. Cada componente responde a una necesidad explícita del caso o a una restricción documentada.
- **Trazabilidad de datos.** Cada registro tiene linaje desde el archivo fuente hasta el KPI, y los registros inválidos se aíslan en una tabla de errores en lugar de descartarse silenciosamente.
- **Defensividad técnica.** Las decisiones (modelo estrella, índices, granularidad, tratamientos de calidad) se justifican aquí para sostener la defensa técnica de 30 minutos.

Alcance entregado

Componente	Estado	Tecnología
Pipeline ETL modular	Implementado	Python 3.11, pandas, SQLAlchemy
Base de datos relacional	Implementada	PostgreSQL 18
Modelo dimensional (estrella)	Implementado	DDL en SQL
Reglas de calidad de datos	7 reglas implementadas	Python + tabla dq_errors
Consultas analíticas y vistas	6 vistas + 8 consultas	SQL
KPIs fintech	7 KPIs documentados	Vistas materializadas
Diccionario de datos final	Completo	Markdown + este documento
Propuesta de evolución AWS	Documentada	Medallion (S3) + Glue + dbt + Redshift + DataZone
Modelo semántico Power BI	Diseñado	Star schema + medidas DAX
Empaquetado y ejecución	Docker Compose	Postgres + ETL container

2. Contexto y entendimiento del caso

2.1 Problema de negocio

TUMIPAY gestiona información de clientes, créditos y pagos en archivos planos dispersos con inconsistencias. El negocio no puede responder con confianza preguntas operativas básicas (cartera vigente, tasa de mora por producto, efectividad por canal) porque cada equipo trabaja con su propia versión de los datos.

La solución debe entregar una fuente única de verdad sobre la cual el área de Riesgo, Comercial y Dirección puedan construir reportes y tomar decisiones.

2.2 Fuentes de información

Archivo	Registros	Granularidad	Observación inicial
clientes.csv	121	1 fila = 1 cliente	Mezcla personas naturales (CC, CE) y jurídicas (NIT)
creditos.csv	262	1 fila = 1 crédito	Relación N:1 con clientes; estados heterogéneos
pagos.csv	1.202	1 fila = 1 transacción de pago	Relación N:1 con créditos; estados duplicados por casing

2.3 Supuestos de negocio adoptados

Estos supuestos resuelven ambigüedades en los datos fuente sin contacto con el negocio. En producción se validarían con el área correspondiente:

- **S1.** Un cliente tipo NIT representa una persona jurídica; por eso puede tener apellidos y fecha de nacimiento nulos sin que sea un defecto de calidad.
- **S2.** Un crédito 'Rechazado' no debe contar en métricas de cartera (no se desembolsó), pero sí queda registrado para análisis de tasa de aprobación.
- **S3.** Un pago 'Reversado' o 'Fallido' no reduce el saldo del crédito; solo los pagos 'Exitoso' impactan la cartera.
- **S4.** La tasa de interés se interpreta como porcentaje mensual nominal según el nombre del campo en el diccionario fuente.
- **S5.** La fecha de corte para KPIs es la fecha máxima observada (data-as-of), no la fecha del sistema, para que los resultados sean reproducibles.
- **S6.** Un crédito está 'en mora' si tiene saldo pendiente y la fecha de corte supera la fecha de vencimiento. El estado textual se conserva y se complementa con un cálculo derivado.

3. Hallazgos de calidad de datos

Antes de diseñar el modelo se realizó análisis exploratorio de los tres archivos para identificar inconsistencias. Estos hallazgos son el insumo principal para las reglas de calidad implementadas en el pipeline.

3.1 Inventario de hallazgos

#	Tabla	Hallazgo	Reg.	Tratamiento
H1	clientes	estado_cliente con casing inconsistente: Activo, activo, EN REVISION	Varios	Corregir (normalizar)
H2	clientes	Emails duplicados entre clientes distintos	15	Marcar (flag)
H3	clientes	numero_documento duplicado	1	Aislar a dq_errors
H4	clientes	apellidos y fecha_nacimiento nulos en NIT	16	Aceptar (S1)
H5	creditos	estado_credito con 'Mora' y 'MORA'	Varios	Corregir (normalizar)
H6	creditos	credito_id duplicado	1	Aislar a dq_errors
H7	creditos	cliente_id no existe en clientes	1	Aislar a dq_errors
H8	creditos	monto_aprobado ≤ 0	1	Aislar a dq_errors
H9	creditos	tasa_interes_mensual nula	1	Aislar a dq_errors
H10	creditos	fecha_desembolso < fecha_solicitud	1	Aislar a dq_errors
H11	pagos	estado_pago: Exitoso, EXITOSO, success	Varios	Corregir (normalizar)
H12	pagos	pago_id duplicado	1	Aislar a dq_errors
H13	pagos	referencia_transaccion duplicada	2	Marcar (flag)
H14	pagos	credito_id no existe en creditos	1	Aislar a dq_errors
H15	pagos	monto_pago ≤ 0 o nulo	3	Aislar a dq_errors

3.2 Estrategia de tratamiento

Tres estrategias se aplican según el tipo de inconsistencia. La elección depende de si el error compromete la integridad estructural (aislar), si es ruido de captura corregible (corregir), o si es señal de negocio que debe llegar al analista (marcar):

Estrategia	Cuando se aplica	Destino
Corregir	Casing, espacios, formatos de fecha, normalización de catálogos	Tabla destino con valor corregido + columna de auditoría
Marcar (flag)	Inconsistencia que no rompe el modelo, pero requiere atención	Tabla destino + columna booleana flag_*

Estrategia	Cuando se aplica	Destino
Aislar	Violación de PK, FK, dominio (montos negativos) o reglas temporales	Tabla dq_errors con motivo y payload original

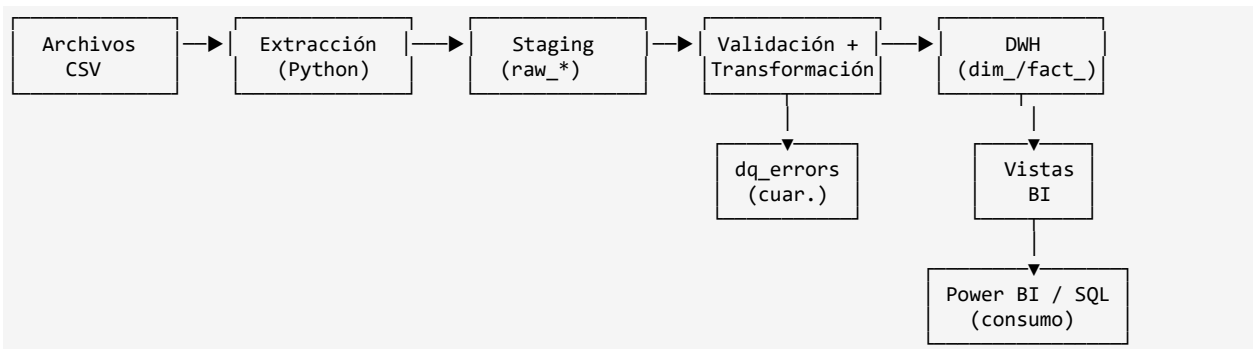
Decisión clave: no se descarta silenciosamente ningún registro. Todo registro inválido tiene trazabilidad en dq_errors con motivo, regla violada y fecha de detección. Esto es crítico para auditoría y para recuperar registros una vez corregidos en el origen.

4. Arquitectura de la solución

4.1 Visión general

La arquitectura sigue un patrón de capas (medallion-like) adaptado al tamaño y criticidad del caso. Se nombran por su función técnica: Raw, Staging, Data Warehouse y Capa Analítica.

Flujo de datos extremo a extremo



4.2 Componentes por capa

Capa 1 — Raw / Ingesta

Los archivos CSV se leen con pandas usando tipos explícitos. No se aplican transformaciones; el único objetivo es traer los datos al ámbito de la base con la mínima pérdida posible. Se carga a tablas raw_clientes, raw_creditos, raw_pagos en el esquema raw.

- Tipos forzados a string para evitar coerciones implícitas que oculten errores.
- Columnas técnicas: _ingested_at (timestamp), _source_file (nombre del archivo).
- Carga full: cada ejecución limpia y reemplaza las tablas raw. En producción se reemplazaría por carga incremental por fecha.

Capa 2 — Staging / Limpieza

Sobre las tablas raw se aplican transformaciones de tipo y normalización en Python, generando stg_clientes, stg_creditos, stg_pagos. Aquí ocurre el trabajo de calidad de datos.

- Conversión a tipos correctos (fechas, enteros, decimales).
- Normalización de catálogos (estados, métodos de pago) a valores canónicos.
- Trimming, lower-casing de emails, validación de patrones.
- Cálculo de columnas derivadas (edad, antigüedad como cliente, etc.).

Capa 3 — Data Warehouse / Modelo dimensional

Esquema dwh con modelo estrella. Las dimensiones son SCD-1 (sobreescritura) para esta entrega; en producción se evaluaría SCD-2 para dimensiones donde el historial sea relevante (segmento del cliente).

- Dimensiones: dim_cliente, dim_producto, dim_canal, dim_tiempo, dim_estado_credito, dim_metodo_pago.
- Hechos: fact_credito (granularidad: un crédito), fact_pago (granularidad: una transacción de pago).
- Llaves subrogadas (surrogate keys) en todas las dimensiones para desacoplar el modelo de identificadores de negocio.

Capa 4 — BI / Consumo

Sobre el DWH se exponen vistas SQL con la lógica de negocio pre-calculada. Estas vistas son el contrato con Power BI: si cambia el modelo subyacente, las vistas absorben el cambio sin romper el dashboard.

- vw_cartera_vigente, vw_morosidad_por_producto, vw_efectividad_canal, vw_cosecha_morosidad, vw_pagos_diarios, vw_kpi_resumen.

4.3 Justificación de decisiones clave

Decisión	Alternativa descartada	Justificación
PostgreSQL 18 como motor	MySQL o SQLite	PostgreSQL soporta JSONB para payloads de errores, CTEs recursivas y dim_tiempo para cosechas, vistas materializadas y extensiones (pg_stat_statements) para tuning. Es estándar fintech en LATAM.
Modelo estrella	Copo de nieve o 3NF	Estrella optimiza lectura analítica (menos joins), es intuitivo para usuarios de negocio y se mapea directo a herramientas BI. La normalización del copo de nieve no aporta valor en este volumen.
Staging separada en BD	Transformar en memoria y cargar directo	Tener staging en base permite reprocesar transformaciones sin re-leer archivos, facilita debugging y habilita validaciones SQL antes de promover al DWH.

Decisión	Alternativa descartada	Justificación
dq_errors centralizada	Logs en archivo	Una tabla permite consultar errores con SQL, generar alertas y construir métricas de calidad. Los logs en archivo se pierden o se rotan.
Carga full vs incremental	CDC desde el inicio	Con 1.585 registros y archivos planos como fuente, full-load es más simple y suficiente. La arquitectura no impide migrar a incremental cuando aparezca la fuente real.
Atomicidad transaccional de load_dwh	Cargas independientes por tabla	Todos los TRUNCATE e INSERT del DWH (dim_tiempo, dimensiones semilla, dim_cliente, fact_credito, fact_pago) ocurren dentro de una única conexión transaccional. Si cualquier paso falla, PostgreSQL hace rollback completo: el DWH nunca queda vacío ni parcialmente cargado. _fetch_sk_map opera dentro de la misma conexión para leer SKs recién insertados antes del commit.
CI/CD con GitHub Actions desde día 1	CI manual o ausente	Pipeline de tres jobs en .github/workflows/ci.yml: unit-tests (pytest sobre transform y validate), docker-build (verifica que la imagen compile), pipeline-smoke (levanta Postgres 18 como service, ejecuta el pipeline completo y valida que etl_runs.status=SUCCESS y dq_errors esté entre 10-15). Demuestra cultura de ingeniería y deja la solución verificable en cada push.
Datos sintéticos commiteados en data/raw/	Generación on-the-fly o externos	Los CSVs son sintéticos generados específicamente para la prueba, no contienen PII real. Commitearlos hace el repo auto-contenido: clonar + docker compose up reproduce el resultado sin pasos adicionales. Esto es crítico para el evaluador.

4.4 Estructura del repositorio

```

fintech-data-platform/
├── README.md                # Documentación principal
├── docker-compose.yml       # Postgres + ETL + pgAdmin
├── Dockerfile               # Imagen del ETL
├── .env.example              # Variables de entorno (sin secretos)
├── .gitignore                # Ignorar .env, __pycache__, etc.
├── requirements.txt          # Dependencias Python
├── pyproject.toml            # Configuración pytest, coverage, ruff
├── .github/
│   └── workflows/
│       └── ci.yml            # CI: unit-tests + docker-build + pipeline-smoke
├── data/
│   ├── raw/                  # CSVs sintéticos commiteados (sin PII real)
│   └── output/                # Reportes generados (opcional)
├── sql/
│   ├── 01_schemas.sql       # Creación de esquemas: raw, stg, dwh, bi, dq
│   ├── 02_ddl_raw.sql        # Tablas raw_*
│   ├── 03_ddl_staging.sql    # Tablas stg_*
│   ├── 04_ddl_dwh.sql        # Dimensiones y hechos
│   ├── 05_ddl_dq.sql         # dq_errors y etl_runs
│   ├── 06_indices.sql        # Estrategia de índices
│   ├── 07_views_bi.sql       # Vistas analíticas
│   ├── 08_roles_grants.sql   # Roles y permisos
│   └── 09_consultas_analiticas.sql # 8 consultas de negocio
├── src/
│   ├── __init__.py
│   ├── main.py                # Orquestador del pipeline
│   ├── config.py              # Carga de configuración
│   ├── logger.py              # Logging estructurado
│   ├── db.py                  # Conexión y helpers SQLAlchemy
│   ├── extract.py             # Lectura de CSVs
│   ├── transform.py           # Limpieza y normalización
│   ├── validate.py            # Reglas de calidad
│   └── load.py                 # Carga atómica a Postgres
├── tests/
│   ├── __init__.py
│   ├── test_transform.py
│   └── test_validate.py
├── docs/
│   ├── Diseno_Solucion_FINTECH.pdf
│   ├── diccionario_datos_final.md
│   ├── arquitectura.png
│   └── modelo_dimensional.png
├── bi/
│   ├── modelo_semantico.md
│   ├── medidas_dax.md
│   └── dashboard.pbix         # (Opcional)

```

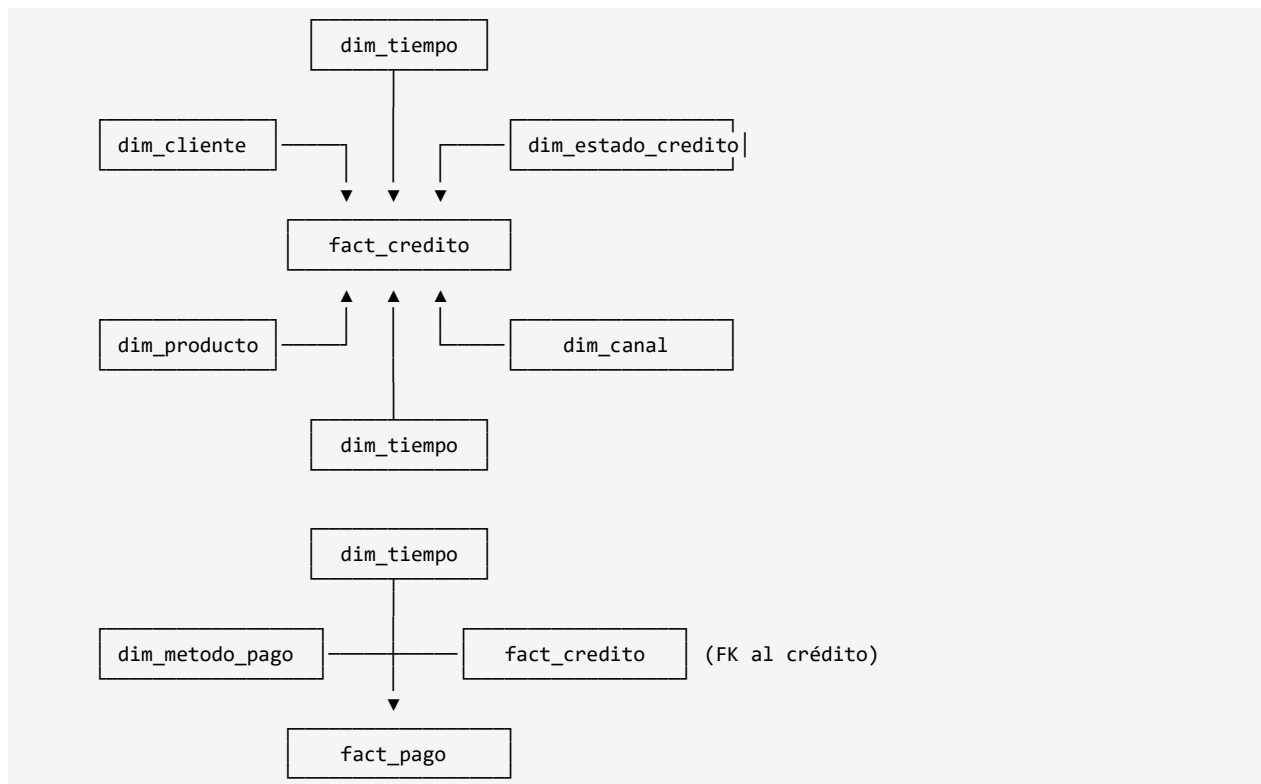
5. Modelo dimensional

5.1 Visión del modelo estrella

El modelo se compone de dos tablas de hechos y seis dimensiones. Las dos tablas de hecho responden a dos preguntas de negocio distintas que no pueden mezclarse en un mismo grano:

- **fact_credito:** ¿Cómo se comporta nuestra cartera de créditos? Una fila por crédito, capturando el ciclo de vida (solicitud, desembolso, estado actual).
- **fact_pago:** ¿Cómo y cuándo nos pagan? Una fila por transacción de pago, capturando el comportamiento de pago.

Diagrama lógico



5.2 Definición de dimensiones

dim_cliente

Campo	Tipo	PK/FK	Descripción
cliente_sk	BIGINT IDENTITY	PK	Llave subrogada autoincremental
cliente_id	VARCHAR(10)	BK	Identificador de negocio del archivo fuente

Campo	Tipo	PK/FK	Descripción
tipo_documento	VARCHAR(5)		CC, CE, NIT
numero_documento	VARCHAR(20)		Número de documento (PII, ver sec. 9)
nombre_completo	VARCHAR(200)		Concatenación nombres + apellidos
email	VARCHAR(150)		Email normalizado (lowercase, trim)
telefono	VARCHAR(20)		Teléfono (PII, ver sec. 9)
ciudad	VARCHAR(60)		Ciudad de residencia
segmento	VARCHAR(20)		Retail, Pyme, Corporativo
estado_cliente	VARCHAR(20)		Activo, Inactivo, Bloqueado, etc. (normalizado)
fecha_registro	DATE		Fecha de alta del cliente
fecha_nacimiento	DATE		NULL para tipo NIT
edad_actual	INT		Calculada respecto a fecha de corte
rango_ingresos	VARCHAR(20)		Bandas (Bajo, Medio, Alto) para análisis
ingresos_mensuales	NUMERIC(14,2)		Ingresos declarados
flag_email_duplicado	BOOLEAN		True si el email aparece en otro cliente
_loaded_at	TIMESTAMP		Auditoría: cuándo se cargó/actualizó

dim_producto, dim_canal, dim_estado_credito, dim_metodo_pago

Dimensiones de baja cardinalidad (3-8 valores cada una). Estructura común:

Campo	Tipo	Descripción
{dim}_sk	INT IDENTITY PK	Llave subrogada
{dim}_codigo	VARCHAR(30)	Valor canónico (clave de negocio)
{dim}_descripcion	VARCHAR(100)	Descripción larga para reportes
categoria	VARCHAR(50)	Agrupador opcional (ej. Producto: Consumo / Empresarial)
activo	BOOLEAN	Permite ocultar valores deprecados sin borrar

dim_tiempo

Dimensión calendario precargada con rango 2023-01-01 a 2027-12-31. Granularidad diaria. Atributos: fecha, año, trimestre, mes, mes_nombre, semana_iso, día_semana,

dia_semana_nombre, es_fin_semana, es_festivo_co (campo poblado con calendario de festividades colombianas).

5.3 Definición de hechos

fact_credito (grano: un crédito)

Campo	Tipo	PK/FK	Descripción
credito_sk	BIGINT IDENTITY	PK	Llave subrogada
credito_id	VARCHAR(10)	BK	Identificador de negocio
cliente_sk	BIGINT	FK	→ dim_cliente
producto_sk	INT	FK	→ dim_producto
canal_sk	INT	FK	→ dim_canal
estado_credito_sk	INT	FK	→ dim_estado_credito
fecha_solicitud_sk	INT	FK	→ dim_tiempo (fecha solicitud)
fecha_desembolso_sk	INT	FK	→ dim_tiempo (fecha desembolso)
fecha_vencimiento_sk	INT	FK	→ dim_tiempo (fecha vencimiento)
monto_aprobado	NUMERIC(14,2)		Medida aditiva
plazo_meses	INT		Plazo del crédito
tasa_interes_mensual	NUMERIC(6,4)		Tasa mensual nominal en %
monto_pagado	NUMERIC(14,2)		Suma de pagos exitosos (calculado en carga)
saldo_actual	NUMERIC(14,2)		monto_aprobado - monto_pagado
dias_mora	INT		max(0, fecha_corte - fecha_vencimiento) si saldo > 0
esta_en_mora	BOOLEAN		dias_mora > 0
fecha_corte	DATE		Fecha de cálculo de las medidas derivadas
_loaded_at	TIMESTAMP		Auditoría

fact_pago (grano: una transacción)

Campo	Tipo	PK/FK	Descripción
pago_sk	BIGINT IDENTITY	PK	Llave subrogada
pago_id	VARCHAR(10)	BK	Identificador de negocio

Campo	Tipo	PK/FK	Descripción
credito_sk	BIGINT	FK	→ fact_credito (degenerada)
cliente_sk	BIGINT	FK	→ dim_cliente (denormalizado para conveniencia BI)
metodo_pago_sk	INT	FK	→ dim_metodo_pago
fecha_pago_sk	INT	FK	→ dim_tiempo
monto_pago	NUMERIC(14,2)		Medida aditiva
estado_pago	VARCHAR(20)		Exitoso, Pendiente, Fallido, Reversado
referencia_transaccion	VARCHAR(30)		Referencia externa
flag_referencia_duplicada	BOOLEAN		True si la referencia aparece en otro pago
_loaded_at	TIMESTAMP		Auditoría

5.4 Granularidad y aditividad

Hecho	Grano	Medida	Aditiva?
fact_credito	1 crédito	monto_aprobado	Sí (sobre cualquier dimensión)
fact_credito	1 crédito	saldo_actual	Semi-aditiva (no sobre tiempo: snapshot)
fact_credito	1 crédito	dias_mora	No aditiva (promediar o max)
fact_credito	1 crédito	tasa_interes_mensual	No aditiva (promediar ponderado por monto)
fact_pago	1 transacción	monto_pago	Sí (sobre cualquier dimensión)

6. KPIs de negocio

Se definen 7 KPIs relevantes para una fintech que gestiona créditos. Cada uno incluye fórmula, fuente y regla de negocio.

6.1 Catálogo de KPIs

KPI-1: Cartera vigente

Atributo	Valor
Definición	Suma del saldo pendiente de todos los créditos activos a una fecha de corte
Fórmula	SUM(saldo_actual) WHERE estado_credito IN ('Activo','Mora','Vencido')
Fuente	fact_credito

Atributo	Valor
Granularidad	Por fecha de corte, producto, canal, segmento, ciudad
Regla de negocio	Excluye créditos Rechazados, Cancelados y Pagados. Mide exposición real.
Audiencia	Dirección, Riesgo, CFO

KPI-2: Tasa de mora (Índice de Cartera Vencida)

Atributo	Valor
Definición	Porcentaje de la cartera con días de mora > 0 sobre la cartera vigente
Fórmula	$\text{SUM}(\text{saldo_actual WHERE dias_mora} > 0) / \text{SUM}(\text{saldo_actual cartera vigente}) \times 100$
Fuente	fact_credito
Granularidad	Por producto, canal, segmento, mes
Regla de negocio	KPI principal de salud del portafolio. Umbral fintech típico: <5% sano, >10% alerta.
Audiencia	Riesgo, Dirección

KPI-3: Ticket promedio por producto

Atributo	Valor
Definición	Monto promedio aprobado por crédito, segmentado por tipo de producto
Fórmula	$\text{AVG}(\text{monto_aprobado}) \text{ WHERE estado_credito} \neq \text{'Rechazado'} \text{ GROUP BY producto}$
Fuente	fact_credito
Granularidad	Por producto, canal, mes de desembolso
Regla de negocio	Permite comparar comportamiento entre productos y detectar cambios de política comercial.
Audiencia	Comercial, Producto

KPI-4: Tasa de aprobación por canal

Atributo	Valor
Definición	Porcentaje de solicitudes desembolsadas sobre total de solicitudes por canal
Fórmula	$\text{COUNT}(\text{credito_id WHERE estado_credito} \neq \text{'Rechazado'}) / \text{COUNT}(\text{credito_id}) \times 100, \text{ GROUP BY canal}$
Fuente	fact_credito

Atributo	Valor
Granularidad	Por canal, producto, mes
Regla de negocio	Mide eficiencia de cada canal (Aliado, Web, App). Permite priorizar inversión comercial.
Audiencia	Comercial, Marketing

KPI-5: Cosecha de morosidad (vintage)

Atributo	Valor
Definición	Porcentaje de créditos de un mes de desembolso que cayeron en mora en los primeros N meses
Fórmula	Por cohorte mes_desembolso: $\text{COUNT}(\text{en mora a M meses}) / \text{COUNT}(\text{cohorte}) \times 100$
Fuente	fact_credito + dim_tiempo
Granularidad	Por mes de desembolso × meses desde desembolso
Regla de negocio	KPI estándar fintech para detectar deterioro en cosechas recientes (señal temprana).
Audiencia	Riesgo (analista senior)

KPI-6: Recaudo mensual

Atributo	Valor
Definición	Suma de pagos exitosos en un período
Fórmula	$\text{SUM}(\text{monto_pago}) \text{ WHERE estado_pago} = \text{'Exitoso'} \text{ GROUP BY mes}$
Fuente	fact_pago
Granularidad	Por mes, método de pago, canal del crédito
Regla de negocio	Métrica de flujo de caja. Excluye reversados y fallidos.
Audiencia	Tesorería, CFO

KPI-7: Tasa de fallo de pago

Atributo	Valor
Definición	Porcentaje de transacciones de pago en estado Fallido o Reversado sobre el total
Fórmula	$\text{COUNT}(\text{pago WHERE estado IN ('Fallido', 'Reversado')}) / \text{COUNT}(\text{pago}) \times 100, \text{ GROUP BY metodo_pago}$
Fuente	fact_pago

Atributo	Valor
Granularidad	Por método de pago, día/semana
Regla de negocio	Detecta problemas operativos con un canal de pago específico (ej. caída de PSE).
Audiencia	Operaciones, Producto

7. Capa BI: vistas y consultas analíticas

7.1 Vistas analíticas (contrato BI)

Vista	Propósito	Refresh
vw_cartera_vigente	Saldo y exposición por cliente/producto/canal a fecha de corte	On-demand
vw_morosidad_por_producto	Tasa de mora segmentada por producto y canal	On-demand
vw_efectividad_canal	Tasa de aprobación y ticket promedio por canal	On-demand
vw_cosecha_morosidad	Matriz de vintage: cohorte de desembolso × meses transcurridos	Materializada (más costosa)
vw_pagos_diarios	Serie temporal de recaudo con desagregación por método	On-demand
vw_kpi_resumen	Vista de una fila con los 7 KPIs a fecha de corte (para tarjetas del dashboard)	Materializada

7.2 Consultas analíticas (mínimo 5 requeridas, se entregan 8)

#	Pregunta de negocio	Tablas involucradas
Q1	Top 10 clientes por exposición vigente	fact_credito, dim_cliente
Q2	Evolución mensual de cartera y mora últimos 12 meses	fact_credito, dim_tiempo
Q3	Ranking de productos por ticket promedio y tasa de mora	fact_credito, dim_producto
Q4	Cosecha de morosidad por mes de desembolso (vintage)	fact_credito, dim_tiempo
Q5	Distribución de pagos por método y porcentaje de fallo	fact_pago, dim_metodo_pago
Q6	Clientes con créditos pero sin pagos exitosos en 60 días	fact_credito, fact_pago, dim_tiempo
Q7	Concentración geográfica de cartera (top 10 ciudades)	fact_credito, dim_cliente
Q8	Tasa de aprobación por canal × segmento del cliente	fact_credito, dim_cliente, dim_canal

Las consultas se entregan en `sql/09_consultas_analiticas.sql` con comentarios explicando el supuesto de negocio, las dimensiones de filtrado típicas y el costo aproximado en plan de ejecución.

8. Capa BI: modelo semántico Power BI

8.1 Modelo semántico propuesto

El modelo se importa desde las vistas analíticas y las dimensiones del DWH, no desde las tablas raw. Esto reduce el riesgo de que un cambio en el modelo físico rompa el dashboard.

Tablas del modelo

Tabla	Origen	Tipo en Power BI	Función
fact_credito	DWH.fact_credito	Hecho (Import)	Granularidad de crédito
fact_pago	DWH.fact_pago	Hecho (Import)	Granularidad de pago
dim_cliente	DWH.dim_cliente	Dimensión	Filtrado por segmento, ciudad, edad
dim_producto	DWH.dim_producto	Dimensión	Filtrado por producto
dim_canal	DWH.dim_canal	Dimensión	Filtrado por canal
dim_tiempo	DWH.dim_tiempo	Dimensión (marcada como tabla de fechas)	Time intelligence
dim_estado_credito	DWH.dim_estado_credito	Dimensión	Filtrado por estado
dim_metodo_pago	DWH.dim_metodo_pago	Dimensión	Filtrado por método

Relaciones

- dim_cliente[cliente_sk] → fact_credito[cliente_sk] (1:N, single direction)
- dim_producto[producto_sk] → fact_credito[producto_sk] (1:N, single direction)
- dim_canal[canal_sk] → fact_credito[canal_sk] (1:N, single direction)
- dim_estado_credito[estado_credito_sk] → fact_credito[estado_credito_sk] (1:N)
- dim_tiempo[fecha_sk] → fact_credito[fecha_desembolso_sk] (1:N, relación activa)
- dim_tiempo[fecha_sk] → fact_credito[fecha_vencimiento_sk] (1:N, relación inactiva → USERRELATIONSHIP)
- dim_tiempo[fecha_sk] → fact_pago[fecha_pago_sk] (1:N, activa)
- dim_metodo_pago[metodo_pago_sk] → fact_pago[metodo_pago_sk] (1:N)
- fact_credito[credito_sk] → fact_pago[credito_sk] (1:N, para drill-down crédito → pagos)

8.2 Medidas DAX principales

```
-- Cartera vigente (KPI-1)
Cartera Vigente =
CALCULATE(
    SUM(fact_credito[saldo_actual]),
    fact_credito[estado_credito] IN {"Activo", "Mora", "Vencido"}
)

-- Tasa de mora (KPI-2)
Tasa Mora % =
DIVIDE(
    CALCULATE(SUM(fact_credito[saldo_actual]), fact_credito[esta_en_mora] = TRUE),
    [Cartera Vigente],
    0
) * 100

-- Ticket promedio (KPI-3)
Ticket Promedio =
CALCULATE(
    AVERAGE(fact_credito[moneto_aprobado]),
    NOT fact_credito[estado_credito] IN {"Rechazado"}
)

-- Tasa de aprobación (KPI-4)
Tasa Aprobacion % =
DIVIDE(
    CALCULATE(COUNTROWS(fact_credito), NOT fact_credito[estado_credito] = "Rechazado"),
    COUNTROWS(fact_credito),
    0
) * 100

-- Recaudo mensual (KPI-6)
Recaudo =
CALCULATE(
    SUM(fact_pago[moneto_pago]),
    fact_pago[estado_pago] = "Exitoso"
)

-- Recaudo mes anterior (para variación)
Recaudo Mes Anterior = CALCULATE([Recaudo], DATEADD(dim_tiempo[fecha], -1, MONTH))
Variacion Recaudo % = DIVIDE([Recaudo] - [Recaudo Mes Anterior], [Recaudo Mes Anterior]) * 100
```

8.3 Diseño conceptual del dashboard

Audiencia y objetivo

Audiencia	Pregunta principal	Decisión que habilita
Dirección / CFO	¿Cómo está la salud financiera de la cartera?	Ajustes de política de crédito; budget de provisiones
Riesgo	¿Qué segmentos/productos/canales están deteriorándose?	Endurecer políticas en cohortes problemáticas
Comercial	¿Qué canales y productos son más rentables/efectivos?	Reasignación de inversión comercial
Operaciones	¿Hay problemas operativos en métodos de pago?	Escalamiento técnico, comunicación a clientes

Estructura propuesta (3 páginas)

Página 1 — Resumen ejecutivo. Tarjetas con los 7 KPIs principales, gráfico de evolución mensual de cartera y mora, mapa de Colombia con cartera por ciudad. Filtros: rango de fechas, producto, segmento.

Página 2 — Análisis de cartera. Distribución por producto, canal y segmento. Top 10 clientes. Detalle por estado del crédito. Matriz de cosecha (vintage) como visual destacada.

Página 3 — Recaudo y pagos. Serie temporal de recaudo, distribución por método de pago, tasa de fallo por método, ranking de días con mayor recaudo.

8.4 Seguridad y acceso (RLS)

Power BI Service expone los reportes con Row-Level Security (RLS) definido en el modelo semántico:

- Rol 'Comercial': solo ve cartera del canal asignado al usuario (filtro por canal).
- Rol 'Riesgo': ve toda la cartera, pero no datos PII (numero_documento, telefono enmascarados).
- Rol 'Dirección': acceso completo a todos los visuales y datos.
- Rol 'Operaciones': solo accede a la página 3 (recaudo y pagos).

La autenticación se delega a Azure AD del cliente. No se manejan usuarios en Power BI directamente.

9. DBA, performance y seguridad

9.1 Estrategia de índices

Los índices se diseñan en función de los patrones de acceso analítico, no operacional. Foco en lecturas de rango por fecha y joins por surrogate key.

Tabla	Índice	Tipo	Justificación
fact_credito	PK (credito_sk)	B-tree implícito	Unicidad y join base
fact_credito	idx_fcredito_cliente (cliente_sk)	B-tree	Drill-down por cliente
fact_credito	idx_fcredito_producto (producto_sk)	B-tree	Filtros por producto
fact_credito	idx_fcredito_fecha_desemb (fecha_desembolso_sk)	B-tree	Reportes por mes de desembolso (cosechas)
fact_credito	idx_fcredito_estado (estado_credito_sk) WHERE estado IN ('Activo','Mora','Vencido')	B-tree parcial	Acelera consultas de cartera vigente; índice pequeño y selectivo
fact_pago	PK (pago_sk)	B-tree	Unicidad
fact_pago	idx_fpago_credito (credito_sk)	B-tree	Join con fact_credito
fact_pago	idx_fpago_fecha (fecha_pago_sk)	B-tree	Series temporales de recaudo
dim_cliente	PK (cliente_sk)	B-tree	Join base
dim_cliente	ux_cliente_bk (cliente_id)	B-tree único	Búsqueda por clave de negocio
dim_tiempo	PK (fecha_sk)	B-tree	Join con todos los hechos

Criterio adicional: no se crean índices sobre dimensiones de baja cardinalidad (producto, canal, estado) más allá de su PK; el optimizador de PostgreSQL prefiere hacer hash join sobre tablas pequeñas. Los índices se mantienen mínimos para no penalizar las cargas.

9.2 Roles y permisos (principio de menor privilegio)

Rol	Permisos	Usuarios típicos
etl_loader	SELECT/INSERT/UPDATE/DELETE en esquemas raw, stg; SELECT/INSERT/UPDATE en dwh	Usuario técnico del pipeline ETL
bi_reader	SELECT en esquemas dwh y bi (vistas). No accede a raw ni stg	Power BI Service, herramientas BI

Rol	Permisos	Usuarios típicos
analyst_readonly	SELECT en dwh y bi. Sin acceso a tablas con PII sin enmascaramiento	Analistas de negocio
dba_admin	ALL en todas las DB	DBA / equipo de plataforma
audit_reader	SELECT en dq_errors, en tabla de auditoría de accesos	Compliance, auditoría interna

9.3 Tratamiento de datos sensibles (PII)

Los siguientes campos se clasifican como PII y reciben tratamiento especial:

Campo	Clasificación	Tratamiento
numero_documento	PII - Alta	Almacenado en claro en dim_cliente con acceso restringido. Vista enmascarada vw_cliente_safe que muestra solo últimos 4 dígitos para analistas.
telefono	PII - Media	Almacenado en claro. Enmascarado en vistas de BI (***-***-1234).
email	PII - Media	Almacenado en claro para join; enmascarado en vistas BI (j***@dominio.com).
nombres, apellidos	PII - Media	Acceso restringido al rol dba_admin y analyst_readonly. bi_reader recibe nombre_corto (primer nombre + inicial apellido).
ingresos_mensuales	Sensible - Financiero	Disponible para Riesgo y Dirección. Para Comercial se expone solo como rango_ingresos (banda).

En PostgreSQL local se aplica el tratamiento por vistas y permisos. En la evolución AWS (sección 10) se usa AWS Lake Formation con column-level permissions y/o KMS-encryption por columna.

9.4 Backup y restore

- Local (Docker): pg_dump diario al volumen montado, retención de 7 backups rotativos. Script bash agendado con cron del contenedor.
- Recuperación punto-en-tiempo (PITR): WAL archiving habilitado, retención 7 días.
- Prueba de restore: script test_restore.sh que levanta una BD efímera, restaura el último dump y ejecuta query de verificación (conteo de filas en fact_credito). Se documenta el tiempo de RTO observado.
- En AWS (sección 10): Aurora PostgreSQL con backups automáticos, snapshots cross-region, retención 35 días.

9.5 Auditoría y trazabilidad

- Cada tabla del DWH incluye columna `_loaded_at` (TIMESTAMP) que registra el momento de inserción/actualización.
- Tabla de auditoría `etl_runs` con: `run_id`, `fecha_inicio`, `fecha_fin`, `status`, `registros_extraidos`, `registros_validos`, `registros_rechazados`, `mensaje`.
- Logs estructurados en JSON (campos: `timestamp`, `level`, `module`, `message`, `run_id`) emitidos por el pipeline. En AWS: CloudWatch Logs con métricas derivadas.
- `pg_stat_statements` habilitado para identificar consultas costosas y planear índices adicionales.

9.6 Performance — consultas críticas y optimización

Consulta	Estrategia
Cartera vigente a fecha de corte	Índice parcial sobre <code>estado_credito</code> . Cache en vista materializada <code>vw_kpi_resumen</code> con refresh diario.
Cosecha de morosidad (matriz vintage)	Vista materializada (<code>vw_cosecha_morosidad</code>). Cálculo costoso (cross-join cohorte × meses), no se rehace en cada consulta.
Recaudo diario últimos 12 meses	Índice sobre <code>fecha_pago_sk</code> . Particionado por mes si el volumen crece > 50M pagos.
Drill-down crédito → pagos	Índice sobre <code>fact_pago_credito_sk</code> . Plan de ejecución verificado con EXPLAIN ANALYZE.

Las vistas materializadas se refrescan al final del pipeline ETL. Esto asegura que el dashboard siempre vea datos consistentes con la última carga, evitando lecturas durante la transformación.

9.7 Atomicidad transaccional de la carga al DWH

Una decisión técnica crítica en la implementación: todos los TRUNCATE e INSERT del pipeline de carga al Data Warehouse (`dim_tiempo`, `dimensiones semilla`, `dim_cliente`, `fact_credito`, `fact_pago`) se ejecutan dentro de una única transacción de PostgreSQL. Esto garantiza que el DWH nunca quede en un estado inconsistente.

El patrón de implementación en `src/load.py` utiliza una única conexión con bloque context manager:

```
with get_connection(engine) as conn:
    truncate_dwh_tables(conn)
    load_dim_tiempo(conn)
    load_dimensiones_semilla(conn)
    load_dim_cliente(conn, df_clientes)
    sk_map_cliente = _fetch_sk_map(conn, 'dim_cliente', 'cliente_id')
    load_fact_credito(conn, df_creditos, sk_map_cliente)
```

```
load_fact_pago(conn, df_pagos)
# commit implícito al cerrar el bloque sin excepciones
```

- Si cualquier operación falla, PostgreSQL hace rollback completo de toda la carga.
- El DWH nunca queda vacío ni parcialmente cargado: o se carga todo, o se mantiene el estado anterior.
- `_fetch_sk_map` opera dentro de la misma conexión para poder leer surrogate keys recién insertadas antes del commit final.
- Esto es esencial para mantener integridad referencial entre dimensiones y hechos durante la carga.

Sin esta atomicidad, un fallo a mitad de la carga (por ejemplo, al insertar `fact_pago` tras haber truncado y poblado `dim_cliente`) dejaría el DWH con dimensiones nuevas pero hechos incoherentes. Power BI mostraría datos inconsistentes. La transacción única elimina esa clase de error por construcción.

9.8 CI/CD con GitHub Actions

El repositorio incluye un pipeline de integración continua en `.github/workflows/ci.yml` que se ejecuta automáticamente en cada push y pull request. El objetivo no es despliegue (esta es una entrega de prueba técnica), sino demostrar cultura de ingeniería: el código se verifica en un entorno limpio y reproducible cada vez que cambia.

Estructura del CI

Job	Propósito	Acciones clave
unit-tests	Validar lógica de transformación y reglas de calidad	Setup Python 3.11, instala requirements + pytest, ejecuta pytest tests/ con cobertura sobre src/
docker-build	Verificar que la imagen del ETL compile sin errores	docker build sobre el Dockerfile del proyecto. Falla si hay errores de sintaxis o dependencias rotas
pipeline-smoke	Validar el pipeline end-to-end en condiciones reproducibles	Levanta postgres:18 como service container, ejecuta el pipeline completo con los CSVs commiteados, valida que <code>etl_runs.status='SUCCESS'</code> y que <code>dq_errors</code> contenga entre 10-15 registros (los hallazgos esperados de la sec. 3.1)

Por qué este CI es importante para la defensa

- Demuestra que el código no solo funciona en la máquina del candidato: se prueba en un runner limpio de GitHub cada vez.

- El job pipeline-smoke es la prueba final: si pasa, significa que un evaluador puede clonar el repo y ejecutar docker compose up con confianza.
- La validación de dq_errors entre 10-15 detecta regresiones funcionales: si alguien rompe una regla de calidad sin darse cuenta, el CI falla.
- Es proporcional al alcance: tres jobs simples, sin sobreingeniería. No hay deploy a AWS ni gates de aprobación porque no aplican al caso.

10. Evolución a producción en AWS

Esta sección describe cómo evoluciona la solución local a un entorno productivo en AWS cuando el volumen, la criticidad o la concurrencia lo justifiquen. No es una migración 1:1: cada componente local se reemplaza por un servicio gestionado equivalente, adoptando además el patrón de arquitectura Medallion (Bronze / Silver / Gold) que es estándar de la industria para data lakes/lakehouses.

La propuesta combina tres pilares: (1) almacenamiento por capas Medallion sobre S3, (2) transformaciones declarativas gobernadas con dbt para la capa analítica, y (3) gobierno federado opcional con AWS DataZone a partir de la maduración del modelo.

10.1 Adopción del patrón Medallion (Bronze / Silver / Gold)

El patrón Medallion organiza el data lake en tres capas con responsabilidades diferenciadas. Esta separación habilita tiering de costos (cada capa con su clase de almacenamiento S3), control de acceso granular (Lake Formation por capa), y trazabilidad clara de calidad de datos: cada capa tiene su propio nivel de confiabilidad declarado.

Capa	Propósito	Equivalente local	Almacenamiento AWS	Formato
Bronze (Raw)	Datos crudos tal cual llegan de la fuente. Inmutable, append-only, fuente de verdad para reprocesamiento.	raw_* en Postgres	S3 Standard-IA (acceso infrecuente) con tiering a Glacier tras 90 días	Parquet, particionado por fecha de ingesta (yyyy/mm/dd)
Silver (Curated)	Datos limpios, tipados, catálogos normalizados. Schema enforced. Lista para consumo por equipos técnicos.	stg_* en Postgres	S3 Standard (acceso frecuente desde Glue, Athena)	Parquet, particionado por dominio y fecha de negocio
Gold (Consumption)	Modelo dimensional listo para BI. Tablas de hechos y dimensiones, vistas analíticas, KPIs pre-calculados.	dwh.* y bi.vw_* en Postgres	Redshift (hot) + S3 Standard para tablas frías vía Spectrum	Nativo Redshift + Parquet en S3

La adopción de Medallion no es por moda. La justificación es concreta: separación de costos por capa (Glacier para Bronze frío vs Standard para Silver activo), separación de gobierno (Lake Formation aplica políticas distintas a cada zona), y trazabilidad de calidad (cada capa tiene su SLA declarado: Bronze = sin garantía, Silver = schema enforced + DQ pass, Gold = curated + tested).

10.2 Arquitectura objetivo en AWS



10.3 Transformaciones con dbt: dónde aplica y dónde no

Para la transformación Silver → Gold se propone dbt con el adapter dbt-redshift. dbt es el estándar de facto para transformaciones SQL gobernadas: versionado en git, tests declarativos,

lineage automático y documentación generada. Sin embargo, dbt no reemplaza a Glue; los complementa. La división de responsabilidades es la siguiente:

Capa de transformación	Herramienta	Justificación
Bronze → Silver	AWS Glue (PySpark o Python Shell)	Lectura de Parquet raw, parsing complejo, schema enforcement, validaciones DQ con AWS Glue Data Quality, escritura particionada. Estas operaciones se expresan mejor en código procedural que en SQL puro.
Silver → Gold	dbt-redshift (modelos SQL)	Construcción del modelo dimensional (dim_*, fact_*) y vistas BI. dbt aporta tests declarativos, lineage column-level, documentación auto-generada (dbt docs) y materialización inteligente (view, table, incremental).
Gold → consumo	Vistas y materializaciones Redshift	Vistas materializadas para KPIs costosos (cosecha de morosidad). Refresh disparado por dbt run al final del pipeline.

Lo que dbt aporta concretamente al proyecto FINTECH:

- Tests declarativos por modelo: `unique(credito_id)`, `not_null(monto_aprobado)`, `relationships(cliente_id → dim_cliente)`. Reemplaza una porción del código Python de validación con declaraciones en YAML.
- Lineage automático: dbt detecta dependencias entre modelos por análisis del SQL (`ref('stg_creditos')`) y genera el grafo. Reemplaza la documentación manual de linaje.
- dbt docs: sitio estático auto-generado con catálogo de modelos, descripciones, tests, lineage visual. Se publica en S3 + CloudFront para consumo de analistas y compliance.
- Materialización flexible: por modelo se decide si es view (siempre fresca), table (snapshot diario) o incremental (solo cambios). Optimiza costos de Redshift.
- Snapshots de cambios (SCD-2 nativa): dbt snapshots permite implementar SCD-2 en `dim_cliente` sin código procedural.

Cuando NO uso dbt: para extracción de fuentes no-SQL, parsing de archivos crudos, validaciones que requieren lógica compleja en Python (regex, ML), o cuando la cantidad de modelos no justifica la curva. Con 2 hechos y 6 dimensiones, dbt es proporcional pero su valor real aparece a partir de 30-50 modelos. La inversión vale porque escala con el negocio sin refactor.

¿Por qué no usar dbt en todas las fases del pipeline?

Una propuesta común — pero técnicamente incorrecta — es usar dbt como herramienta única para todo el pipeline (ingesta, Bronze, Silver, Gold). Esta sección documenta por qué la división Glue/dbt es superior, anticipando la pregunta más probable en la defensa técnica.

La premisa de partida es entender qué es dbt en realidad: un compilador de SQL con tests, materialización y lineage. No es un motor de procesamiento (corre sobre el warehouse subyacente), no es una herramienta de ingesta, no es un orquestador, y no es un lenguaje de propósito general. dbt es excelente en su nicho; el error técnico es estirar ese nicho a fases donde el SQL no es la herramienta correcta.

1. Bronze → Silver: dbt no maneja parsing y schema enforcement. Para transformar Parquet crudo a Parquet limpio se requiere schema inference sobre archivos posiblemente malformados, parsing de campos complejos (fechas en múltiples formatos, números con separadores regionales, JSON embebido en strings), manejo de archivos corruptos y validaciones DQ a nivel archivo. PySpark resuelve esto con funciones nativas y modos como PERMISSIVE; SQL puro lo hace con CASE WHEN anidados frágiles. En el caso FINTECH, normalizar valores como 'Exitoso', 'EXITOSO', 'success' es trivial en Glue; en dbt es posible pero el archivo todavía no tiene schema antes de parsearlo, y dbt no parsea archivos.

2. La ingesta misma no es territorio de dbt. Antes de Bronze hay un paso crítico: traer los datos desde la fuente hasta S3. Esto requiere DMS para replicación OLTP con CDC, Lambda o Glue para archivos vía SFTP/API, Kinesis o MSK para streaming. dbt no realiza ninguna de estas operaciones. Si la pregunta es '¿dónde está la ingesta?', la respuesta nunca puede ser dbt.

3. Validaciones que requieren Python no se expresan bien en SQL. dbt tiene tests built-in (unique, not_null, accepted_values, relationships) y permite tests custom en SQL/Jinja. Pero hay validaciones donde SQL es subóptimo: regex complejo (formato de email, dígito de verificación del NIT colombiano que usa multiplicaciones y módulo), validaciones contra fuentes externas (listas de sancionados, catastro tributario), detección de patrones temporales con lógica condicional. En PySpark estas validaciones se implementan con UDFs claras y testeables; en dbt quedan como macros de Jinja con SQL crudo, frágil y difícil de mantener.

4. dbt depende de un motor SQL: sin motor, no hay dbt. Este es el punto más sutil pero el más importante. dbt no es un sistema autónomo; es una capa de orquestación de SQL sobre un warehouse (Redshift, Snowflake, BigQuery, Postgres, Databricks). Si se quiere correr dbt sobre Bronze, hay que apuntar un motor SQL a Bronze. Las opciones — Athena/Trino, Redshift Spectrum, Databricks SQL — tienen tradeoffs concretos: Athena es lento sobre Parquet sin particionado óptimo; Spectrum obliga a cargar datos crudos al warehouse, contaminándolo con basura; Databricks ya no es AWS puro. El antipattern resultante es usar Redshift como 'área de trabajo de limpieza', lo cual es caro e ineficiente.

5. El costo computacional se invierte cuando se carga Bronze a Redshift. Cuando dbt corre sobre Redshift, el compute es Redshift, que cuesta por hora encendido o por RPU en Serverless.

Transformaciones pesadas tipo Bronze→Silver consumen el cluster que necesitan los usuarios BI. Glue PySpark, en cambio, es serverless puro: solo se paga por segundos de ejecución, auto-escala según volumen, y no compete con Redshift por recursos. Para 1.585 registros no importa; para 100M sí, y muchísimo.

6. dbt no maneja datos no estructurados. Si en el futuro llegan a FINTECH PDFs de contratos para extraer cláusulas (Textract + Lambda), imágenes de cédulas para validación KYC (Rekognition), o audios de call center para análisis de sentimiento (Transcribe + Comprehend), dbt no toca nada de esto. Solo trabaja con datos ya estructurados en un warehouse.

Donde dbt sí es imbatible es en la fase Silver → Gold: una vez los datos están limpios, tipados y en formato analítico, dbt aporta valor sin competencia. Joins complejos entre hechos y dimensiones, cálculos de negocio (saldo actual, días de mora, cosechas), materialización inteligente (view/table/incremental según el caso), tests sobre el modelo dimensional, documentación auto-generada, lineage column-level, y SCD-2 nativa vía snapshots. Esto es exactamente el trabajo de construir un DWH dimensional, y dbt no tiene competidor real en este nicho.

Regla de oro defendible

dbt es la herramienta correcta cuando los datos ya tienen estructura tabular limpia y la transformación es expresable en SQL. Antes de ese punto — ingesta, parsing, schema enforcement, validaciones de Python — Glue o Lambda son superiores porque tienen el compute y el lenguaje correcto.

Cuando sí existe 'dbt para todo': el caso Snowflake/BigQuery

Hay equipos que efectivamente usan dbt desde la ingesta. Esto es posible en stacks donde el warehouse absorbe la responsabilidad de ingesta vía servicios nativos: Snowflake con Snowpipe (ingesta declarativa desde S3), BigQuery con Dataflow o Storage Transfer, Databricks con Auto Loader. En esos contextos, dbt puede tocar desde Bronze porque el motor del warehouse ya hizo la ingesta sin código procedural.

En AWS con Redshift no existe ese equivalente nativo de igual nivel. COPY desde S3 es la opción más cercana, pero requiere que el archivo ya esté en formato compatible — no resuelve parsing complejo ni schema inference. Por eso la propuesta para FINTECH mantiene Glue como herramienta de ingesta y transformación de bajo nivel: es la forma defendible en AWS, no en otros stacks.

Resumen visual: herramienta correcta por fase

Fase	Operación típica	Herramienta correcta	Por qué no dbt
Ingesta	Replicar de OLTP, leer S3, consumir Kafka	DMS, Glue, Kinesis, Lambda	dbt no tiene capacidad de ingesta
Bronze (landing)	Recibir Parquet crudo, particionar por fecha	S3 + Glue Catalog	dbt no escribe archivos crudos
Bronze → Silver	Parsing, schema enforcement, normalización, DQ a nivel archivo	Glue PySpark / Python Shell	SQL es subóptimo para parsing y validaciones Python
Silver → Gold	Modelo dimensional, joins, cálculos, SCD-2	dbt-redshift (sweet spot)	Aquí sí es la herramienta correcta
Gold → consumo	Vistas BI, agregaciones, refresh	dbt + Redshift materialized views	Aquí también es correcta
Orquestación	Disparar Glue→dbt→refresh, retry, alertas	Step Functions, MWAA, Airflow	dbt no orquesta sistemas externos
Observabilidad	Logs, métricas, alertas	CloudWatch, Datadog, Grafana	dbt emite artifacts, no es plataforma de obs
Catálogo de negocio	Productos de datos, ownership, glossary	DataZone, Glue Catalog	dbt docs es catálogo técnico, no de negocio

dbt es excelente en su nicho (Silver→Gold y Gold→consumo) y subóptimo fuera de él. Conocer la diferencia es lo que hace un punto importante a tener en consideración.

10.4 Gobierno de datos: Lake Formation y AWS DataZone

El gobierno se aborda en dos niveles, adoptados en fases distintas según madurez de la organización:

Fase 1 — Lake Formation (desde el día 1)

- Control de acceso fino a nivel de tabla, columna y fila sobre el data lake (Bronze, Silver, Gold en S3).
- Permisos por LF-tag: tag DataClassification (Public, Internal, Confidential, PII) propagado a todos los recursos. Las políticas se escriben sobre tags, no sobre nombres de tablas, lo que escala.
- Integración nativa con IAM Identity Center: los roles de Power BI, analistas y procesos ETL se autentican vía SSO corporativo.

- Audit logs centralizados en CloudTrail para todo acceso al data lake.

Fase 2 — AWS DataZone (mes 6+, según madurez)

DataZone resuelve un dolor que aparece a partir de cierta escala: descubrimiento y federación de productos de datos. Permite que cada dominio de negocio (Riesgo, Comercial, Operaciones) publique sus datasets con ownership, SLA, glossary y métricas de calidad visibles a consumidores. Es el componente que habilita un modelo Data Mesh.

- Dominios de negocio definidos: Cartera, Recaudo, Riesgo, Comercial. Cada uno con un data steward responsable.
- Catálogo de productos de datos: cartera_vigente, cosecha_morosidad, recaudo_diario, etc. Publicados por el dominio dueño, suscritos por consumidores con workflow de aprobación.
- Glossary de negocio compartido: definiciones canónicas de 'cartera vigente', 'tasa de mora', 'cliente activo' para eliminar ambigüedad entre áreas.
- Integración con Glue Catalog y Lake Formation: DataZone no reemplaza, agrega una capa de gobierno federado encima sin refactor del almacenamiento.

DataZone no se adopta el día 1. Para 1.585 registros y un equipo pequeño es sobreingeniería. Su valor aparece cuando hay múltiples dominios productores y consumidores con necesidad de gobierno federado. La consigna de Tumipay pide explícitamente no sobredimensionar; por eso DataZone se posiciona como roadmap, no como MVP.

10.5 Mapeo componente local → AWS (actualizado)

Componente local	Servicio AWS	Justificación
Archivos CSV / fuente operacional	AWS DMS + SCT → S3 Bronze (Parquet)	DMS replica desde OLTP con full-load + CDC sin afectar el origen. SCT convierte esquemas heterogéneos. Parquet reduce 70%+ almacenamiento y acelera lecturas columnares.
Capa raw_* en Postgres	S3 Bronze (Parquet)	Inmutable, particionado por fecha de ingesta. Glue Catalog para descubrimiento. Tiering a Glacier tras 90 días para reducir costo.
Capa stg_* en Postgres	S3 Silver (Parquet) + AWS Glue Job	Glue (Python Shell para volumen bajo, PySpark para volumen alto) aplica schema enforcement, normalización de catálogos y DQ. Resultado en Silver con Glue Catalog.
Validaciones Python en validate.py	AWS Glue Data Quality + dbt tests	Glue DQ para validaciones a nivel de archivo (completitud, formato). dbt tests para reglas de negocio sobre modelos analíticos (unique, relationships).

Componente local	Servicio AWS	Justificación
Capa dwh.* en Postgres (dim/fact)	dbt-redshift models → Redshift Gold	dbt construye dim_* y fact_* declarativamente desde Silver. Tests automáticos. Lineage column-level. Materializaciones table/incremental.
Capa bi.vw_* (vistas analíticas)	dbt models + Redshift materialized views	Vistas regulares como modelos dbt 'view'. KPIs costosos (cosecha) como materialized views refrescadas al final del run.
Tabla dq_errors	S3 Quarantine + tabla Redshift + dbt failure storage	Errores físicos en S3 (Bronze quarantine zone, retención larga). dbt test failures persisten en tabla dedicada para reportes de calidad.
Diccionario manual (.md)	dbt docs + Glue Catalog + DataZone (fase 2)	dbt docs genera el catálogo de modelos automáticamente. Glue Catalog indexa Silver. DataZone agrega capa de productos de datos con ownership.
Logs Python (archivo/stdout)	CloudWatch Logs + dbt artifacts en S3	Logs estructurados centralizados. dbt artifacts (run_results.json, manifest.json) almacenados en S3 para observabilidad y debugging.
Cron del contenedor	EventBridge Scheduler + Step Functions	Step Functions orquesta el pipeline: Glue Job (Bronze→Silver) → dbt run → dbt test → refresh views. Reintentos y fallback nativos.
Roles y permisos PostgreSQL	IAM + Lake Formation + Redshift roles	IAM gobierna acceso a servicios. Lake Formation: permisos column/row level en S3 con LF-tags. Redshift roles para el DWH.
Credenciales en .env	AWS Secrets Manager	Rotación automática, audit logs, integración nativa con Glue, Lambda y dbt-redshift profile.
Backups locales (pg_dump)	Aurora snapshots + S3 versioning + Glacier	Aurora para metadatos. S3 versioning automático + lifecycle a Glacier para retención larga (compliance regulatorio).
Power BI local (.pbix)	Power BI Service + Gateway / DirectQuery a Redshift	Servicio gestionado con RLS, refresh programado y distribución. Gateway si Redshift está en VPC privada.

10.6 Diseño Redshift específico (capa Gold)

Mi experiencia diseñando DWH en Redshift para Bancolombia aplica directamente a la capa Gold:

- Tablas fact con DISTKEY = cliente_sk (alta cardinalidad, distribuye uniformemente; mantiene join localmente con dim_cliente que se hace ALL para tablas pequeñas).
- SORTKEY compuesto en fact_credito: (fecha_desembolso_sk, estado_credito_sk). El primero acelera filtros temporales; el segundo refina por estado dentro del rango.
- Dimensiones pequeñas (dim_producto, dim_canal, dim_estado_credito, dim_metodo_pago) con distribución ALL para evitar redistribución en joins.

- WLM con cola separada: 'etl' (memoria mayor, timeout largo, para dbt run) y 'bi' (mayor concurrencia, query timeout corto, para Power BI).
- Redshift Spectrum sobre S3 Silver para análisis ad-hoc sobre data histórica fría, sin cargarla a Redshift. Reduce el footprint del cluster.
- VACUUM y ANALYZE delegados a Auto Workload Management; monitor manual mensual sobre tablas críticas (fact_credito, fact_pago).
- dbt models con materialización 'incremental' para fact_pago (alto volumen, append-mostly): solo procesa pagos nuevos, no rehace todo el histórico.

10.7 Seguridad en AWS

- VPC privada para Redshift, Aurora y endpoints Glue. Sin IP pública. VPC endpoints (Gateway para S3, Interface para Glue/Secrets Manager) para no salir a internet.
- Acceso a Power BI vía AWS PrivateLink + Power BI Gateway en EC2 dentro de la VPC.
- KMS para cifrado at-rest de S3 (CMK por capa: Bronze, Silver, Gold), Redshift, Aurora y Secrets Manager. Llaves CMK gestionadas por el cliente para control de rotación.
- Lake Formation con LF-tags por clasificación (PII, Confidential, Internal, Public). Las políticas se aplican sobre tags, no sobre nombres.
- CloudTrail habilitado en todas las cuentas. Logs centralizados en cuenta de seguridad (Landing Zone).
- GuardDuty + Security Hub para detección de amenazas y postura de seguridad.
- IAM con principio de menor privilegio. Roles por servicio (no usuarios programáticos compartidos). dbt corre con un rol dedicado con permisos solo sobre Silver y Gold.
- Tag mandatorio costcenter en todos los recursos para imputación de costos a áreas de negocio.

10.8 Escalabilidad y decisiones de capacidad

Dimensión	Volumen actual	Cambio a 100x	Cambio a 1000x
Almacenamiento	1.585 filas, KB	100K filas, MB	Mantener Redshift; S3 con Intelligent Tiering / Glacier para Bronze frío
Compute ETL (B→S)	Python local	Glue Python Shell (DPU bajo)	Glue PySpark con auto-scaling de DPUs
Compute T (S→G)	Python + SQL	dbt-redshift (Serverless 8 RPU)	dbt-redshift en cluster ra3 + Concurrency Scaling para builds paralelos
Compute DWH	PostgreSQL local 2vCPU	Redshift Serverless 8 RPU	Redshift ra3 cluster + Concurrency Scaling

Dimensión	Volumen actual	Cambio a 100x	Cambio a 1000x
Concurrencia BI	1-2 usuarios	10-50 usuarios	100+ usuarios: Power BI Premium + Redshift Concurrency Scaling
Latencia carga	Full diaria	Incremental cada hora	Streaming: Kinesis + Glue Streaming + Redshift COPY incremental

El criterio para escalar no es el tamaño del dato sino la latencia que tolera el negocio y la concurrencia de usuarios BI. Una fintech regulada típicamente tolera datos D-1; eso permite arquitecturas más simples y baratas que las de tiempo real. Medallion + dbt + DataZone es el camino evolutivo natural, adoptado por fases según madurez.

11. Gobierno de datos y calidad

11.1 Reglas de calidad implementadas (≥5 requeridas, se entregan 7)

#	Regla	Tabla	Tratamiento
DQ-1	Unicidad de llave primaria (cliente_id, credito_id, pago_id)	Las 3	Aislar duplicados a dq_errors
DQ-2	Integridad referencial (créditos→cliente, pagos→crédito)	creditos, pagos	Aislar huérfanos a dq_errors
DQ-3	Normalización de catálogos (estado_cliente, estado_credito, estado_pago)	Las 3	Corregir con mapping table
DQ-4	Validación de dominio (montos > 0, plazo > 0, tasa entre 0-10%)	creditos, pagos	Aislar fuera de rango a dq_errors
DQ-5	Coherencia temporal (fecha_desembolso ≥ fecha_solicitud)	creditos	Aislar inconsistencias
DQ-6	No nulidad de campos críticos (monto, fechas clave, tasa)	creditos, pagos	Aislar registros incompletos
DQ-7	Detección de duplicados de negocio (email cliente, referencia pago)	clientes, pagos	Marcar con flag, no aislar

11.2 Modelo de la tabla dq_errors

```
CREATE TABLE dq.dq_errors (
  error_id      BIGSERIAL PRIMARY KEY,
  run_id        VARCHAR(50)  NOT NULL,
  tabla_origen  VARCHAR(50)  NOT NULL, -- raw_clientes, raw_credits, raw_pagos
  regla_id      VARCHAR(20)  NOT NULL, -- DQ-1, DQ-2, ...
  regla_descripcion VARCHAR(200) NOT NULL,
  bk_origen     VARCHAR(100), -- clave de negocio del registro
  payload_original JSONB      NOT NULL, -- el registro completo
  severidad     VARCHAR(10)  NOT NULL, -- ERROR, WARNING
  fecha_deteccion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_dq_errors_run ON dq.dq_errors(run_id);
CREATE INDEX idx_dq_errors_regla ON dq.dq_errors(regla_id);
```

11.3 Métricas de calidad

Cada ejecución del pipeline emite un resumen de calidad que se almacena en etl_runs y se visualiza en la página 4 del dashboard:

- Total registros leídos por tabla.
- Total registros validados exitosamente.
- Total registros aislados a dq_errors (con desglose por regla).
- Tasa de calidad = válidos / leídos × 100.
- Tiempo de ejecución por etapa (extract, transform, validate, load).

11.4 Lineage / linaje de datos

El linaje se documenta en el diccionario de datos final (sección 12 y archivo separado). Para esta entrega es documental; en la evolución AWS se automatiza con AWS Glue Data Catalog + lineage features (column-level lineage). Alternativas: OpenLineage + Marquez/DataHub.

12. Diccionario de datos del modelo final

Esta sección presenta un resumen del diccionario. El archivo completo se entrega como docs/diccionario_datos_final.md en el repositorio.

12.1 Resumen de tablas

Esquema	Tabla	Propósito	Granularidad
raw	raw_clientes	Datos brutos del CSV de clientes	1 fila = 1 cliente
raw	raw_creditos	Datos brutos del CSV de créditos	1 fila = 1 crédito
raw	raw_pagos	Datos brutos del CSV de pagos	1 fila = 1 pago
stg	stg_clientes	Clientes con tipos correctos y catálogos normalizados	1 fila = 1 cliente válido
stg	stg_creditos	Créditos limpios	1 fila = 1 crédito válido
stg	stg_pagos	Pagos limpios	1 fila = 1 pago válido
dwh	dim_cliente	Dimensión cliente	1 fila = 1 cliente
dwh	dim_producto	Catálogo de productos crediticios	1 fila = 1 producto
dwh	dim_canal	Catálogo de canales	1 fila = 1 canal
dwh	dim_tiempo	Calendario	1 fila = 1 día
dwh	dim_estado_credito	Catálogo de estados	1 fila = 1 estado
dwh	dim_metodo_pago	Catálogo de métodos de pago	1 fila = 1 método
dwh	fact_credito	Hecho de créditos	1 fila = 1 crédito
dwh	fact_pago	Hecho de pagos	1 fila = 1 pago
bi	vw_cartera_vigente	Vista analítica	Por crédito vigente
bi	vw_morosidad_por_producto	Vista analítica	Por producto
bi	vw_efectividad_canal	Vista analítica	Por canal
bi	vw_cosecha_morosidad	Vista materializada	Por cohorte × meses
bi	vw_pagos_diarios	Vista analítica	Por día × método
bi	vw_kpi_resumen	Vista materializada	Una fila con 7 KPIs
dq	dq_errors	Registros rechazados con motivo	1 fila = 1 violación
dq	etl_runs	Auditoría de ejecuciones	1 fila = 1 corrida del pipeline

El diccionario detallado por campo (tipo, descripción funcional, regla de transformación, sensibilidad) se entrega en el archivo Markdown del repositorio para facilitar mantenimiento y diffs en git.

13. Plan de ejecución del proyecto

Esta sección describe el plan de trabajo concreto para construir la entrega final. Estimación: 8-10 horas efectivas.

13.1 Cronograma propuesto

Fase	Tiempo	Entregables
1. Diseño (este documento)	Completado	PDF con arquitectura, modelo, KPIs, AWS Medallion + dbt
2. Setup repositorio + Docker	30 min	GitHub repo, docker-compose.yml, .env.example, requirements.txt, pyproject.toml
3. Scripts DDL (sql/01-08)	1.5 h	Esquemas, DDL raw/stg/dwh/dq, índices, roles, vistas
4. Pipeline ETL Python	3 h	src/ con extract, transform, validate, load (atómico), main; logger; config
5. Consultas analíticas	1 h	sql/09_consultas_analiticas.sql comentadas
6. Pruebas básicas	30 min	tests/test_transform.py, test_validate.py
7. CI GitHub Actions	30 min	.github/workflows/ci.yml con unit-tests + docker-build + pipeline-smoke
8. README + diccionario .md	1.5 h	README.md completo, diccionario_datos_final.md
9. Power BI	1 h	dashboard.pbix con 2-3 visuales clave, capturas
10. Validación end-to-end	30 min	Ejecución limpia desde docker compose up, CI verde en GitHub

13.2 Definición de 'hecho' (Definition of Done)

1. docker compose up levanta Postgres 18 y ejecuta el pipeline sin errores.
2. El pipeline produce métricas en etl_runs y registros en dq_errors coherentes con los hallazgos documentados (sección 3).
3. La carga al DWH es atómica: si falla a media carga, hace rollback completo y el DWH queda en su estado anterior.
4. Las 6 vistas BI retornan filas no nulas.
5. Las 8 consultas analíticas se ejecutan sin error y retornan resultados defendibles.
6. pytest tests/ pasa en local con cobertura sobre src/.

7. CI en GitHub Actions pasa los tres jobs: unit-tests, docker-build y pipeline-smoke (con `etl_runs.status=SUCCESS` y `dq_errors` entre 10-15 registros).
8. El README permite a un evaluador clonar y ejecutar la solución siguiendo las instrucciones literalmente.
9. El diccionario de datos final está completo (todas las tablas, todos los campos).
10. Los archivos del repositorio se subieron a GitHub con commit history coherente (no un único 'initial commit').

14. Mejoras futuras y limitaciones reconocidas

14.1 Limitaciones de la entrega actual

- SCD-1 en todas las dimensiones. Si el segmento de un cliente cambia, se pierde el histórico. Mitigación futura: SCD-2 en `dim_cliente`.
- Carga full-load. No es viable para volúmenes operacionales reales. Mitigación: CDC con DMS o triggers.
- Pruebas unitarias mínimas (cobertura básica de transform y validate). Mitigación: ampliar a pytest con fixtures de datos y >70% de cobertura.
- Sin orquestación. El pipeline se ejecuta como script único. Mitigación: Airflow / Step Functions con DAG por capa.
- Calidad de datos sin alertas. `dq_errors` se llena pero no notifica. Mitigación: integración con Slack/Email vía SNS.
- No hay versionado del modelo. Cambios al DDL no están bajo control de versiones formal. Mitigación: Flyway o Liquibase.

14.2 Roadmap evolutivo (post-entrega)

Fase	Objetivo	Tecnologías
Mes 1-2	Migrar a AWS adoptando arquitectura Medallion (Bronze/Silver/Gold) sobre S3 + Redshift. Mantener el modelo lógico.	AWS DMS, Glue, S3, Redshift, Power BI Gateway
Mes 3	Introducir dbt-redshift para transformaciones Silver→Gold. Migrar lógica SQL de Python a modelos dbt con tests declarativos y dbt docs.	dbt Core / dbt Cloud, dbt-redshift adapter
Mes 4	Implementar SCD-2 en <code>dim_cliente</code> vía dbt snapshots. Orquestación con Step Functions. Glue Data Quality para validaciones a nivel de archivo.	dbt snapshots, Step Functions, EventBridge, Glue DQ
Mes 5	Modelo de scoring crediticio (ML) sobre el DWH para predicción de mora. Feature engineering desde Silver.	SageMaker + Feature Store + endpoints

Fase	Objetivo	Tecnologías
Mes 6+	Gobierno federado con AWS DataZone: dominios de datos (Cartera, Recaudo, Riesgo, Comercial), catálogo de productos de datos con ownership y SLA. Habilita modelo Data Mesh.	AWS DataZone + Lake Formation + Glue Catalog

14.3 Riesgos identificados

Riesgo	Probabilidad	Impacto	Mitigación
Datos PII expuestos a usuarios sin necesidad	Media	Alto	RLS + vistas con enmascaramiento. KMS en AWS.
Crecimiento de cartera supera capacidad del modelo single-region	Baja	Medio	Diseño cloud-ready desde el día 1 facilita migración.
Cambios en archivos fuente sin notificación	Alta	Medio	Schema validation en extract. Alert si cambia estructura.
Errores en lógica de cálculo de mora	Media	Alto	Pruebas unitarias específicas + revisión cruzada con Riesgo.

Este diseño busca demostrar criterio de Data Platform Lead aplicado a un caso concreto de fintech: una solución proporcional al problema, con trazabilidad total, defendible técnicamente y con una evolución cloud realista basada en experiencia previa con clientes similares (Bancolombia, Terpel, Claro).

— Fin del documento —